

IntraFuzz: Coverage-Guided Intra-Enclave Fuzzing for Intel SGX Applications

Jinhua Cui, Qiao Peng, Yiwen Yao, Ke Ye, Jiliang Zhang*

College of Semiconductors (College of Integrated Circuits), Hunan University, Changsha 410082, China

{jhcui, pqlush, ywyao, keye, zhangjiliang}@hnu.edu.cn

Abstract—Intel SGX is susceptible to intra-enclave software vulnerabilities. Existing automated bug-finding methods primarily focus on fuzzing enclave boundaries for SGX applications in simulated, rather than *actual* hardware-protected enclaves. This limits the ability to identify potential security violations originating from within SGX application code. This paper presents INTRAFUZZ, the first system that enables efficient fuzzing of SGX applications inside *actual* hardware enclaves. We evaluated INTRAFUZZ with 21 real-world SGX applications, running on Intel Xeon scalable processors with up to 256 GB of enclave page cache. INTRAFUZZ successfully detected all vulnerabilities in SGX application code previously identified by the state-of-the-art tool EnclaveFuzz, as well as 6 previously undiscovered vulnerabilities. These results highlight the importance of hardware-based fuzzing in securing SGX applications.

I. INTRODUCTION

Today's commercial processors natively support trusted execution environments (TEEs) to safeguard data in use from unauthorized access [1]–[3]. A prime example is Intel's Software Guard eXtensions (SGX) [4]–[7], which establishes a hardware-isolated environment, called *enclave*, specifically designed to be non-introspectable to prevent data leaks.

Despite hardware-enforced protections, SGX applications remain vulnerable to software flaws that can undermine system security [8]–[10]. Many are written in C/C++ languages, which are prone to memory vulnerabilities like buffer overflow and use-after-free. For instance, a heap or stack overflow in mbedTLS-SGX can be exploited to leak the entire enclave memory by manipulating untrusted inputs [10]. Even seemingly minor issues like null pointer dereference can become critical in SGX environments, as the untrusted OS controls the zero address [11]. Moreover, SGX-specific malware poses additional threats, such as stealing RSA keys from other enclaves [12] or encrypting sensitive data to restrict access [13].

The fundamental cause of these vulnerabilities often stems from developer-introduced flaws, either intentional or unintentional, and untrusted third-party libraries. For example, developers unfamiliar with SGX requirements may inadvertently introduce vulnerabilities into SGX applications [14]–[17]. On the other hand, legacy code is frequently ported into SGX enclaves [18]–[23] with limited validation, which inherits existing security flaws (e.g., OpenSSL library) or introduces new enclave-specific risks. These issues highlight the need for efficient detection mechanisms tailored to SGX applications.

Recent studies have introduced several tools to detect vulnerabilities in SGX applications [10], [11], [24]–[33]. Among them, fuzzing is more effective for vulnerability detection, as it automatically generates diverse inputs to rigorously test

applications. Nevertheless, existing SGX fuzzing tools still exhibit notable limitations: 1) Lack of *actual* hardware fuzzing. Existing approaches like SGXFuzz [24] and EnclaveFuzz [11] perform fuzzing exclusively in simulated enclave environments, which lack essential SGX-specific protections and features, such as remote attestation and sealing/unsealing. This limits detection accuracy and scope. Further, SEnFuzzer [26] runs programs within actual SGX enclaves but collects only selective runtime data, which leads to insufficient analysis and potential false positives. 2) Incomplete application behavior analysis. Current methods focus primarily on enclave boundary interfaces (e.g., ECALLs or/and OCALLs) [10], [11], [24] and often overlook internal application bugs. For performance and security purposes [9], [34], SGX applications are often designed to use a single ECALL-OCALL pair to ensure that execution completes without intermediate exits or re-entries. As a result, the absence of runtime context information makes analyzing the application code challenging. 3) High overhead of crash detection on SGX. Exception handling in SGX enclaves is resource-intensive, which complicates efficient crash detection in hardware-based fuzzing. Current tools avoid this issue, as they typically run in simulated environments without SGX exception handling costs. This affects the detection speed and accuracy on actual hardware.

In this paper, we introduce INTRAFUZZ, an automated framework for full-process fuzzing of SGX applications directly within actual hardware enclaves. We adapt a coverage-guided fuzzing framework for SGX, which leverages a modified library OS as the runtime. This runtime transparently manages interactions between the target application and the fuzzing engine. INTRAFUZZ employs an intra-enclave tracer to monitor each fuzzing process and iteratively generates refined inputs through seed mutation. This process updates the corpus with high-quality seeds to enable efficient crash detection and vulnerability identification during program execution. Further, it incorporates an efficient process management mechanism based on in-enclave shared memory, which allows rapid runtime information exchange across processes. To handle hardware exceptions caused by program crashes, we set up a signal handling trampoline to promptly capture crashes and analyze them by parsing enclave stack memory traces.

We evaluate INTRAFUZZ on 21 real-world SGX applications running on Intel Xeon scalable processors, with up to 256 GB of enclave page cache. INTRAFUZZ successfully detected all vulnerabilities in SGX application code previously reported by the state-of-the-art tool EnclaveFuzz, along with 6 previously undiscovered vulnerabilities.

Detection Scope. This work focuses on fuzzing SGX application code within actual enclaves, rather than the host-enclave

*Corresponding author: Jiliang Zhang

boundary in simulated environments, which prior studies have extensively explored. Thus, it departs from existing methods and complements efforts to secure hardware enclaves.

Our contributions are summarized as follows:

- We conduct a systematic analysis of differences and challenges between simulated and *actual* hardware enclaves.
- We propose the first full-process intra-enclave fuzzing framework for SGX applications, called INTRAFUZZ, which enables internal tracers to monitor security violations in target application code.
- We evaluate INTRAFUZZ with 21 open-source projects configured to run with up to 256 GB of enclave page cache, and uncover 9 vulnerabilities in them, including 6 previously undiscovered ones.

II. BACKGROUND

A. Hardware-enforced Enclaves with Intel SGX

Intel Software Guard eXtensions (SGX) [35] is a set of secure instructions integrated into Intel x86 processors. It protects sensitive data and code within an isolated execution environment, known as an enclave, from the rest of the system.

Enclave Entry/Exits. SGX introduces specialized instructions to control the interaction between the enclave and non-enclave code. In particular, the EENTER instruction manages entry into the enclave (ECALLs), while EEXIT controls the exit (OCALLs). The processor validates the ECALL by checking that it points to an authorized entry located in the enclave’s metadata. A predefined wrapper function is used to handle the OCALL request and securely return to the enclave. However, note that for performance and security consideration [9], [34], developers often utilize a single ECALL-OCALL pair for SGX applications to guarantee that execution completes without extra exits or re-entries.

Exception Handling in SGX Enclaves. When an exception occurs within an enclave, the hardware first triggers an Asynchronous Enclave Exit (AEX) to hand over control to the OS. After the OS receives the exception, it sends the corresponding signal to the host process. The host process re-enters the enclave via EENTER, and the exception handler within the enclave reads the CPU state in the Saved State Area (SSA) and executes the default or user-defined handling code. Once completed, the enclave exits via EEXIT, and the host process then resumes normal execution of the enclave via ERESUME. This mechanism ensures the secure saving and restoring of the enclave’s state during exception handling.

B. Coverage-guided Fuzzing

Fuzzing is an automated technique for identifying software vulnerabilities by repeatedly executing the target program with diverse inputs [36]. Coverage-guided fuzzing refines this process by utilizing coverage feedback to select inputs that explore more new execution paths. Tools like Honggfuzz [37] compute coverage based on metrics such as executed instructions, branches, and unique basic blocks and retain test cases that uncover new code as good seeds. Prior SGX fuzzing methods [11], [24]–[26] primarily perform in non-SGX environments, where feedback is collected outside the enclave. This often misses critical runtime states within enclaves and hence causes overlooked vulnerabilities and false positives. Moreover, these methods mainly focus on testing enclave

boundaries, such as ECALLs, OCALLs, and system calls. In contrast, our work emphasizes identifying vulnerabilities within the security-critical application code inside enclaves. This complements existing methods, collectively offering a more comprehensive strategy for securing SGX enclaves.

III. ANALYSIS OF SGX ENCLAVE FUZZING

A. Simulated vs. Actual Hardware Enclaves

Intel offers a software simulation environment that enables SGX functionality on machines without native SGX hardware support [38]. In simulation mode, SGX instructions are fully emulated through software. Since fuzzing depends on executing target programs and monitoring their runtime behavior to uncover vulnerabilities, the discrepancies between simulated and actual hardware environments can substantially diminish the reliability and effectiveness of fuzzing results.

D1. Security Gaps. Unlike actual hardware-based SGX, which provides strong security guarantees, simulated enclaves lack these protections that leaves them vulnerable to attacks. As a result, attackers can bypass software-based protections through various methods to steal sensitive data. This means fuzzing SGX applications in actual hardware enclaves can more accurately evaluate their security, as simulated environments fail to replicate real-world threats and risks.

D2. Library Discrepancies. Simulated and real hardware environments differ significantly in implementation. In hardware mode, SGX APIs interact with hardware features through dedicated libraries, such as `sgx_trts.lib`. In contrast, simulation mode relies on alternative libraries, such as `sgx_trts.lib_sim`, to emulate this functionality. Similarly, `sgx_tcrypto_opt.lib`, a hardware-optimized cryptographic library, provides enhanced performance on actual SGX hardware, while `sgx_tcrypto.lib` is a basic library intended for simulation environments.

D3. Functional Gaps. Certain operations, such as remote attestation, require hardware support and are unavailable in simulation mode. As a result, fuzzing conducted in a simulated environment produces less accurate results and may fail to fully reflect the effectiveness of tests on actual hardware. In contrast, hardware-based fuzzing provides more reliable and representative outcomes. While SEnFuzzer [26] introduces an on-device fuzzing framework, it still relies on extracting ECALL and OCALL data to construct a fuzzing harness, without running the application throughout SGX enclaves.

Consequently, we emphasize that fuzzing SGX applications on actual hardware is essential for uncovering vulnerabilities that may not manifest in simulated environments and for accurately reflecting real-world attack surfaces.

B. Challenges of Fuzzing on Actual Intel SGX CPUs

C1. Constraints on Intra-enclave Multi-process Fuzzing. Intel SGX imposes strict limits on the size of protected memory, typically 128 MB or 256 MB [4]. As such, the number of enclaves that can be instantiated simultaneously on a single platform is constrained by the memory footprint of each enclave. In addition, fuzzing, which typically employs a multi-process model for efficiency, demands real-time execution and monitoring across multiple processes. However, hardware enclaves lack native support for multitasking within a single

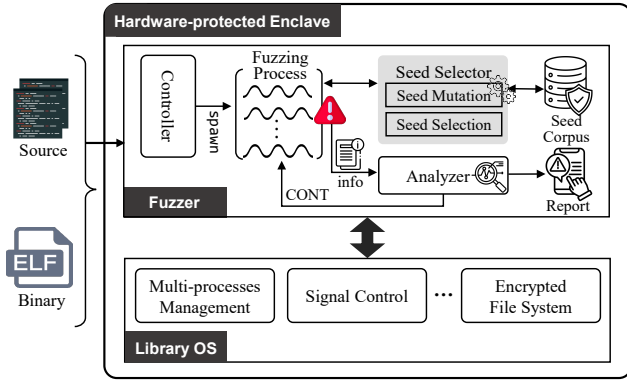


Fig. 1: INTRAFUZZ Overview

enclave. Running multiple enclaves for parallel fuzzing processes drastically reduces efficiency due to the high overhead of inter-enclave communication.

C2. Obstacles to Intra-enclave Monitoring. The security design of Intel SGX ensures that enclaves are isolated from one another and non-introspectable to the outside world. Consequently, conventional monitoring tools cannot directly access or inspect an enclave’s internal operations. This inherent opacity poses significant challenges for detecting program crashes, tracking coverage information, and applying other fuzzing techniques within the SGX ecosystem.

C3. Costly Enclave I/O Operations. Communication across enclave boundaries is limited to the interfaces defined in an EDL file, namely ECALLs and OCALLs. This increases complexity and delays input mutation and output reporting during fuzzing. Further, since the enclave cannot directly access the file system, all file operations must be carried out through OCALLs. This affects the overall performance of the tests, particularly when frequent file reads and writes are required during fuzzing.

IV. INTRAFUZZ APPROACH

A. Overview

As illustrated in Figure 1, INTRAFUZZ enables intra-enclave fuzzing of SGX applications within actual SGX environments. The process starts with preparing the instrumented source code or binary file and initial seeds. At the beginning of fuzzing, the *seed selector* mutates the initial seed. The *controller* clones worker threads and spawns enclave processes to execute the target application with mutated inputs. When a crash occurs, the *analyzer* parses the crashed stack and generates a detailed report. During each iteration, the seed selector updates a seed corpus with high-quality inputs that either trigger vulnerabilities or explore new code paths. The *library OS* provides basic services, such as system call interfaces and an encrypted file system, to facilitate intra-enclave fuzzing. It employs in-enclave shared memory for process management, which enables precise control over the target program’s execution state and efficient delivery of runtime information. INTRAFUZZ further integrates an exception-based monitoring mechanism to detect unexpected program behaviors and utilizes enclave stack traces for crash diagnostics.

To overcome the three challenges outlined in Section III-B, we 1) harness an SGX library OS to enable multi-process fuzzing inside a single enclave on SGX v2 with up to 256

GB of enclave private memory, which effectively supports an unlimited number of concurrent enclaves; 2) propose an intra-enclave multi-process management module to monitor program behaviors and gather runtime information; 3) incorporate an encrypted file system into SGX enclaves to enable secure and efficient seed file access during fuzzing.

B. Efficient Multi-process Management

Since the fuzzer process requires control over the target process and the collection of its runtime information, communication between the processes is essential. However, the fuzzer and target processes are isolated from each other due to the enforcement of multi-domain Software Fault Isolation (SFI) technology [39]. To address this limitation, we design a shared memory mechanism between the processes. Each block of shared memory is 2048 bytes in size and divided into three fields: *exec* (256 bytes), *info* (80 bytes), and *stack* (the left space). The *exec* field records execution status, the *info* captures details about abnormal signals and the *stack* stores information for stack backtracking.

As shown in Figure 2, the target process communicates its runtime state to the fuzzer process via shared memory. During initialization, the fuzzer creates N shared memory blocks, where N is the number of parallel processes specified by the user. To be specific, the main fuzzing process first clones N worker threads. Each worker thread then creates a child process using *spawn* to execute the target program. To organize this setup, the parent worker thread and its child process are grouped into a *workgroup*, with the parent thread’s *TID* indexing the corresponding memory block to obtain runtime information. This design adheres to SGX’s memory constraints while ensuring that each process operates securely within actual enclaves.

C. Enclave Crash Monitoring

When a crash occurs in the SGX application code, INTRAFUZZ must interpose the target process to collect runtime context information for detailed analysis. However, since each crash causes the current process to exit the SGX enclave—triggering an asynchronous enclave exit (AEX) exception—it is essential to restore control flow to the target process. To address this, we propose a signal handling based implementation, as shown in Figure 3. During the instrumentation of target program, a custom signal handler is registered (❶). When an exception occurs, this handler is invoked to collect crash details and store runtime context information into shared memory (❷). The analyzer subsequently reads this information, identifies the type of crash, and determines whether it represents a previously unseen issue (❸). If a new crash is detected, the analyzer generates a report and saves the seed (❹).

After completing the analysis, the analyzer writes *cont* signal to the *exec* field in shared memory to inform the target program to continue execution (❺). The target program then resumes from the point of the crash, which will trigger the same signal (e.g., SIGSEGV) and eventually exiting due to the unhandled exception. At the end of each iteration, the controller spawns a new child process to start the next iteration. This loop continues until a termination condition is reached, at which point the fuzzing process ends.

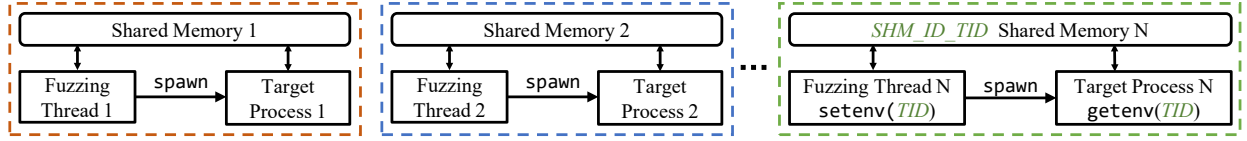


Fig. 2: Efficient Process Management via Shared Memory

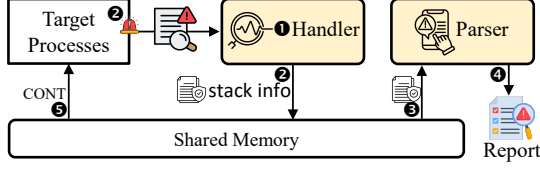


Fig. 3: Exception-based Monitoring Workflow

D. Enclave Crash Analyzing

When the target process crashes, gathering program details is necessary for analysis. However, pinpointing the root cause of a crash can be challenging. To address this, we introduce an adaptive enclave stack memory parsing mechanism within the analyzer. Upon receiving a signal, it dynamically analyzes runtime information using memory mappings, call stack traces, and instruction disassembly. Specifically, it extracts essential details from the signal structure (`siginfo_t`) and the program counter (PC), which are stored in the `info` field of shared memory. It then leverages the `libunwind` library to trace the call stack and parses the `/proc/[pid]/maps` file to retrieve memory mapping details. Finally, we leverage Capstone engine to disassemble instructions in memory. Based on these above, INTRAFUZZ delivers real-time, detailed insights into the execution path and instruction-level information.

Spawn-based process creation introduces the challenge of identical instruction addresses mapping to different memory addresses, which makes the address alone insufficient to uniquely identify crashes. Despite each child process operating in a separate virtual space, the layout of code segments within the virtual address space is consistent across processes. Thus, crashes at the same instruction exhibit identical lower 12 bits (in-page offset) of the virtual address, which reflects the instruction’s position in the code segment. Based on this, we define crash uniqueness using four key attributes: (i) the signal type, (ii) the lower 12 bits of the instruction address, (iii) the current instruction, and (iv) a hash value of the call stack.

V. IMPLEMENTATION

We implemented INTRAFUZZ prototype by extending honggfuzz [37], a lightweight, feedback-driven fuzzer, to work within advanced SGX runtime systems such as Occlum [18]. INTRAFUZZ retains the core components of honggfuzz, including the mutation algorithm, and test case scheduling. To enable intra-enclave fuzzing, we developed several key modules, such as multi-process management, signal control and enclave stack memory parsing. Due to SGX’s isolation constraints, the `fork` system calls in honggfuzz are not supported within a single enclave. To resolve this, we modified the controller to use `spawn` for creating enclave processes. Moreover, the target program and initial seed are stored in the encrypted file system to facilitate the fuzzing.

VI. EVALUATION

In this section, we evaluated INTRAFUZZ on real-world SGX applications to demonstrate its effectiveness in detecting vulnerabilities. We also compare INTRAFUZZ with existing state-of-the-art SGX vulnerability detection techniques and analyze their methodologies. Finally, we highlight the vulnerabilities newly uncovered in the target SGX applications.

A. Evaluation Settings

Experiment Setup. The evaluation was conducted on a server equipped with four Intel Xeon Platinum 8444H CPUs, 512 GB of RAM, and 256 GB of enclave page cache, running Ubuntu 20.04 with the 5.15.0-124-generic kernel. Each project was fuzzed for over 24 hours.

Tested SGX Applications. We selected 20 widely recognized open-source projects with notable star ratings on GitHub. Table I presents the fuzzing results for these SGX applications, including the size, coverage, and verified bugs. The specific versions of these applications were adapted to be compatible with INTRAFUZZ.

B. Vulnerability Detection

INTRAFUZZ detected hundreds of crashes across 7 applications during fuzzing. Since a single bug can lead to multiple crashes, deduplication is required. We applied an intra-enclave adaptive stack trace parsing mechanism to identify unique crashes, followed by manual analysis. This process revealed 9 distinct and verified bugs, as shown in Table I.

Detail of Reports. Table II shows a detailed summary of the bugs identified by INTRAFUZZ and other works. Two buffer overflow vulnerabilities were discovered in `SGXCryptoFile` and `cave`, both caused by input file sizes exceeding expected limits. These vulnerabilities were uniquely detected by our approach, as previous methods failed to preserve the original semantics of the applications, which is essential for effective fuzzing. In `sgx-wallet`, a double-free vulnerability was found. In `webassemblyEnclave`, a logic error was identified, where incorrect control flow leads to unintended execution branches. In `sgx-kmeans`, an out-of-bound access error was observed due to insufficient validation before data allocation. Detailed explanations of these vulnerabilities are provided in Section VI-D. For clarity, Table II excludes crash counts associated with boundary interface issues for `SGXFuzz` and `EnclaveFuzz`.

To validate the effectiveness of detecting vulnerabilities in the remote attestation process, which is not supported in simulated enclaves, we introduced a fault into a confidential function in the program. Experimental results demonstrated that INTRAFUZZ successfully identified the vulnerability.

Coverage. Table I shows that INTRAFUZZ achieves comparable coverage to `EnclaveFuzz`. While `EnclaveFuzz` analyzes the source code without considering contextual semantics, INTRAFUZZ, by fully preserving the context of SGX applications, results in improved coverage. Specifically, `EnclaveFuzz` achieves 14.8% coverage, whereas INTRAFUZZ reaches 17.7%.

TABLE I: Fuzzing Results of INTRAFUZZ and EnclaveFuzz

SGX Application	Version	Size	Coverage		Verified Bugs	
			EnclaveFuzz	IntraFuzz	EnclaveFuzz	IntraFuzz
SGXCryptoFile_Encrypt [40]	92f3cd6	520.7KB	18.5%	22.7%	1*	0
SGXCryptoFile_Decrypt [40]	92f3cd6	520.7KB	—	13.1%	—	1
sgx-gwas [41]	24e59f3	1.8MB	0.05%	0.08%	3 [◇]	0
SGX-DFC [42]	d3f7e47	657.4KB	1.2%	13.7%	0	0
sgx-kmeans [43]	8ab6035	618.0KB	16.8%	24.3%	5 [◇]	2
sgx-wallet [44]	8d15df1	59.0KB	36.9%	8.2%	1* + 1 [◇]	1
sgx-GuessTheNumber [45]	69bf5b3	577.6KB	26.5%	23.1%	1 [◇]	0
SGX-AES-256 [46]	b0d87a8	707.6KB	—	2.9%	—	0
sgx-nbench [47]	799f0fc	486.8KB	22.3%	36.2%	1* + 9 [◇]	1
SwitchlessSGXSan [48]	d0778b9	474.0KB	—	22.9%	—	0
SampleSGXSan [49]	71c4227	497.0KB	20.7%	14.6%	3 [◇]	0
sgxvnp [52]	6e18300	1.8MB	2.4%	0.2%	1 [◇]	0
webassemblyEnclave_Add [51]	2281ac9	1.3MB	15.3%	3.8%	8 [◇]	1
webassemblyEnclave_Mul [51]	2281ac9	1.3MB	—	3.6%	—	1
sgxvnp [52]	7e61a63	469.0KB	26.0%	12.5%	5 [◇]	0
ucmalloc-sgx [53]	3e2cdeb	492.5KB	6.2%	20.0%	—	0
mex-sgx [54]	429473e	424.1MB	5.3%	63.4%	—	0
sgxBankApp [55]	8637537	796.8KB	28.6%	2.8%	0	0
cave [56]	bb8ef3a	1.9MB	5.5%	3.6%	3 [◇]	1
MatrixChainOrder [57]	1952bce	497.1KB	3.9%	64.1%	0	0
Attestation-test [58]	Crafted	463.5KB	—	16.0%	—	1
Total Bugs & Average Coverage			14.8%	17.7%	3*	9

* indicates bugs found within security-critical application code. [◇] means bugs located at the host-enclave boundary. — denotes fuzzing failures under the original configurations.

TABLE II: Unique Crashes Identified across Different Works

SGX Application	Unique Crashes in SGX Application		
	SGXFuzz	EnclaveFuzz	IntraFuzz
SGXCryptoFile [40]	0	1	1
sgx-kmeans [43]	0	0	2
webassemblyEnclave [51]	0	0	2
cave [56]	0	0	1
sgx-wallet [44]	1	1	1
sgx-nbench [47]	1	1	1
Total crashes found	2	3	8

TABLE III: Comparison to Existing Methods

Name	Technique	Target	Is Testing Conducted within Actual Enclaves?
TEERex [28]	Symbolic Execution	Enclave Boundary	✗
COIN [10]	Symbolic Execution	Enclave Boundary	✗
SymGx [30]	Symbolic Execution	ECALLs & OCALLs	✗
Pandora [31]	Symbolic Execution	ECALLs & OCALLs	✗
SGXFuzz [24]	Fuzzing	ECALLs	✗
SEnFuzzer [26]	Fuzzing	ECALLs & OCALLs	✗
FuzzSGX [25]	Fuzzing	ECALLs & OCALLs	✗
EnclaveFuzz [11]	Fuzzing	ECALLs & OCALLs	✗
Emilia [27]	Fuzzing	System Calls	✗
IntraFuzz (this work)	Fuzzing	SGX Application Code	✓

✗ indicates fuzzing conducted entirely within simulated enclave environments. ✗ means that SGX applications run inside actual enclaves, while the fuzzing process is performed outside the enclave. ✓ refers to fuzzing conducted entirely within actual, hardware-protected enclaves.

Responsible Disclosure. When a bug is detected and manually verified, we adhere to a 90-day disclosure timeline and notify the affected vendor via email or repository issue.

C. Comparison with State-of-the-art Works

We compare advanced SGX vulnerability detection tools with INTRAFUZZ based on their detection targets and fuzzing environments, as summarized in Table III. Unlike existing tools that primarily focus on boundary interfaces such as ECALLs, OCALLs and syscalls, INTRAFUZZ targets intra-enclave application code. This focus enables it to uncover vulnerabilities that prior methods might overlook and expose critical code segments previously unexamined. INTRAFUZZ is the first to implement full-process fuzzing directly on SGX hardware. This offers comprehensive monitoring of the actual runtime behavior within hardware enclaves. In reproducing EnclaveFuzz, we observed that it requires manual source code

modifications for instrumentation. In contrast, INTRAFUZZ automates this process.

D. Case Study

We showcase representative examples of each bug type identified by INTRAFUZZ.

```

1 int decryptFile(const char* input, const char* output)
2 {
3     ...
4     size_t decMessageLen = fsize - (SGX_AESGCM_MAC_SIZE +
5     SGX_AESGCM_IV_SIZE);
6     unsigned char *decMessage = (unsigned char *) malloc
7     ((decMessageLen+1)*sizeof(unsigned char));
8     sgxDcryptFile(message, fsize, decMessage, decMessageLen);
9     decMessage[decMessageLen] = '\0';
10    fwrite(decMessage, decMessageLen, 1, ofp);
11 }

```

Listing 1: Stack Overflow in CryptoFile

Stack Overflow. Several crashes caused by buffer overflow were found, with SGXCryptoFile as a notable example. As shown in Listing 1, if the input size (fsize) is smaller than SGX_AESGCM_MAC_SIZE+SGX_AESGCM_IV_SIZE, the calculation of decMessageLen results in an underflow (line 4) and produces an unexpected large value decMessageLen. This value is then passed to malloc (line 6), which causes excessive memory allocation. If the allocation succeeds, operations like writing to decMessage (line 10) will exceed the allocated buffer. This leads to memory corruption and hence may overwrite critical memory areas. More severely, this vulnerability may enable attackers to perform arbitrary writes, which compromise the confidentiality and integrity of the enclave.

```

1 cell eval(cell x, environment * env)
2 {
3     if (x.type == Symbol)
4         return env->find(x.val)[x.val];
5     ...
6 }

```

Listing 2: Logical Execution Vulnerability in CAVE

Logical Execution. Listing 2 illustrates a logical execution vulnerability in cave. The behavior of eval, which includes

multiple branches, can be directly influenced by the input data. This vulnerability allows attackers to manipulate the input and alter the program's execution flow within the enclave.

```

1 int* k_means(float** data, int n, int m, int k, float t,
2             float** centroids)
3 {
4     ...
5     for (h = i = 0; i < k; h += n / k, i++) {
6         if (!centroids) {
7             c[i] = (float *) calloc(m, sizeof(float));
8         }
9         /* pick k points as initial centroids */
10        for (j = m; j-- > 0; c[i][j] = data[h][j]);
11    }
12    ...
13    for (i = 0; i < k; i++) {
14        float distance = 0;
15        for (j = m; j-- > 0; distance += pow(data[h][j] -
16        c[i][j], 2));
17    }
18 }

```

Listing 3: Out-of-bound Access in SGX-KMEANS

Out-of-bound Access. Out-of-bound access occurs when a program accesses data outside its intended boundaries, which may lead to memory corruption, crashes, or unpredictable behavior. In Listing 3, the issue arises when `data[h][j]` is inadequate for assignment to `c[i][j]`. The operation `pow(data[h][j] - c[i][j], 2)` may fail due to the improper assignment. Specifically, if `data[h][j]` is not properly validated before the assignment, it may result in out-of-bound access when subtracting `c[i][j]`. This mis-assignment causes the program to access memory beyond the allocated bounds, which finally triggers undefined behavior or memory corruption.

VII. DISCUSSION

INTRAFUZZ automatically detects memory corruption vulnerabilities in Intel SGX applications. However, some memory vulnerabilities may not cause crashes. For example, a missing bounds check in the handling of the TLS heartbeat [59] extension leads to an out-of-bound read across the enclave boundary, which ultimately reveals up to 64 KB of memory data to a connected client or server without triggering a crash. While sanitizers [60], [61] can address these issues, implementing them within the SGX environment requires much engineering effort, which we leave for future work. Moreover, several solutions for SGX compatibility have been proposed [18], [19], but we cannot perform vulnerability detection on these toolchains, as our work primarily focuses on discovering vulnerabilities in SGX application code itself. For such toolchains, fuzzing would likely require the generation of specialized fuzzing harnesses.

VIII. RELATED WORK

A. Fuzzing for SGX Applications

SGXFuzz [24] simulates key SGX execution conditions and uses the kAFL fuzzer along with the Nyx snapshot-based fuzzing engine to recreate critical enclave contexts outside the SGX environment. FuzzSGX [25] is a fuzzer based on input and program mutations. It applies static analysis and symbolic execution to construct the host application and modifies both the application and seed inputs iteratively to evaluate the enclave program. SEnFuzzer [26] extracts ECALL and OCALL information from the EDL files to create specialized fuzzing

drivers. Despite executing the enclave program within an actual SGX environment, the fuzzer runs outside the enclave and only focuses on the ECALL and OCALL interfaces. EnclaveFuzz [11] presents a multi-dimensional structure-aware fuzzing framework that extracts input structures from enclave source code and analyzes their inter-dependencies to generate fuzz harnesses. Emilia [27] intercepts system call return values to reproduce potential Iago attacks [34] through fuzzing. However, it operates outside actual SGX environments and is limited to the return values, which results in reduced coverage. In summary, existing SGX fuzzing methods analyze the enclave boundaries. However, none of them perform full-process fuzzing within actual SGX enclaves, leaving real vulnerabilities undetected, as demonstrated by INTRAFUZZ.

B. Symbolic Execution for SGX Applications

TEERex [28] takes a binary file as input, identifies ECALLs through static analysis, and then uses symbolic exploration to detect vulnerabilities during execution. COIN [10] identifies security vulnerabilities in enclave interfaces (ECALLs/OCALLs) and performs symbolic execution on binary files by parsing interface information from EDL files. SymGX [30] introduces a global transformation graph and context-sensitive analysis model, GSTG-CAP, for simulating SGX applications. It records execution behaviors and drives symbolic execution for vulnerability detection. Pandora [31] uses the angr framework with SGX-specific semantic augmentation. It extracts the enclave's internal state to guide symbolic execution, with plugins designed to detect different vulnerabilities.

C. Others Techniques for Shielding SGX Enclaves

SgxMonitor [32] introduces a security tracking mechanism akin to Intel PT, which implements provenance analysis on SGX. It monitors the target enclave's operating status to ensure it aligns with expectations and identifies any discrepancies. SGXRacer [33] detects controlled data races in enclave code by analyzing shared variables and thread interleavings to ensure proper synchronization. It employs a two-phase approach: identifying shared and lock variables, followed by reentrancy-aware, lockset-based data race detection.

IX. CONCLUSION

We introduce INTRAFUZZ, a new system for efficiently fuzzing SGX applications within actual hardware-protected enclaves. Using an internal tracer, INTRAFUZZ monitors intra-enclave processes to detect security violations with mutated inputs. Our results show INTRAFUZZ identified 9 vulnerabilities across 21 real-world SGX applications on actual Intel SGX processors, including 6 vulnerabilities missed by simulation-based tools like EnclaveFuzz. These findings highlight the importance of actual hardware-based fuzzing for comprehensive security analysis of SGX applications.

X. ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (Grant No. U24A20289, 62402169), the Key R&D Projects in Hunan Province (Grant No. 2024JK2011), the Natural Science Foundation of Hunan Province (Grant No. 2023JJ40160), the Excellent Young Scholar Project of the Hunan Provincial Department of Education (Grant No. 23B0036), the CCF-Huawei Populus Grove Fund (CCF-HuaweiTC202402).

REFERENCES

- [1] Intel, “Microsoft azure confidential computing with intel software guard extensions,” <https://www.intel.com/content/www/cn/zh/developer/articles/technical/microsoft-azure-confidential-computing-with-intel-sgx.html>, 2018.
- [2] J. Associates, “The critical need for confidential computing,” <https://www.intel.cn/content/dam/www/central-libraries/cn/zh/documents/the-critical-need-for-confidential-computing.pdf>, 2020.
- [3] “Alibaba cloud ecs bare metal instance supports intel sgx,” https://www.alibabacloud.com/zh/product/ebm?_p_lc=1&spm=a2c5t.1069566.
- [4] F. McKeen, I. Alexandrovich, A. Berenzon, C. V. Rozas, H. Shafi, V. Shanbhogue, and U. R. Savagaonkar, “Innovative instructions and software model for isolated execution.” *HASP@ISCA*, 2013.
- [5] B. C. Xing, M. Shanahan, and R. Leslie-Hurd, “Intel® software guard extensions (intel® sgx) software support for dynamic memory allocation inside an enclave,” *HASP*, pp. 1–9, 2016.
- [6] Intel, “Intel software guard extensions sdk-documentation,” <https://software.intel.com/en-us/sgx-sdk/documentation>.
- [7] V. Costan, “Intel sgx explained,” *IACR Cryptol, EPrint Arch*, 2016.
- [8] J. Van Bulck, D. Oswald, E. Marin, A. Aldoseri, F. D. Garcia, and F. Piessens, “A tale of two worlds: Assessing the vulnerability of enclave shielding runtimes,” in *ACM SIGSAC CCS*, 2019, pp. 1741–1758.
- [9] J. Cui, J. Z. Yu, S. Shinde, P. Saxena, and Z. Cai, “SmashEx: Smashing sgx enclaves using exceptions,” in *ACM SIGSAC CCS*, 2021, pp. 779–793.
- [10] M. R. Khandaker, Y. Cheng, Z. Wang, and T. Wei, “COIN attacks: On insecurity of enclave untrusted interfaces in sgx,” in *ASPLOS*, 2020, pp. 971–985.
- [11] L. Chen, Z. Li, Z. Ma, Y. Li, B. Chen, and C. Zhang, “EnclaveFuzz: Finding vulnerabilities in sgx applications,” in *NDSS*, 2024.
- [12] M. Schwarz and M. Lipp, “When good turns evil: Using intel sgx to stealthily steal bitcoins,” *Black Hat Asia*, 2018.
- [13] M. Marschalek, “The wolf in sgx clothing,” *Bluehat IL*, 2018.
- [14] M. Schwarz, S. Weiser, and D. Gruss, “Practical enclave malware with intel sgx,” in *DIMVA*, 2019, pp. 177–196.
- [15] J. Lee, J. Jang, Y. Jang, N. Kwak, Y. Choi, C. Choi, T. Kim, M. Peinado, and B. B. Kang, “Hacking in darkness: Return-oriented programming against secure enclaves,” in *USENIX Security*, 2017, pp. 523–539.
- [16] S. Van Schaik, A. Kwong, D. Genkin, and Y. Yarom, “Sgxax: Dfc string pattern matching library for sgx application,” <https://github.com/sparkly9399/SGX-DFC>, 2020.
- [17] N. Weichbrodt, A. Krumus, P. Pietzuch, and R. Kapitza, “Asyncshock: Exploiting synchronisation bugs in intel sgx enclaves,” in *ESORICS*, 2016, pp. 440–457.
- [18] Y. Shen, H. Tian, Y. Chen, K. Chen, R. Wang, Y. Xu, Y. Xia, and S. Yan, “Occlum: Secure and efficient multitasking inside a single enclave of intel sgx,” in *ASPLOS*, 2020, pp. 955–970.
- [19] C.-C. Tsai, D. E. Porter, and M. Vij, “Graphene-SGX: A practical library OS for unmodified applications on SGX,” in *USENIX ATC*, 2017, pp. 645–658.
- [20] J. Cui, S. Shinde, S. Sen, P. Saxena, and P. Yuan, “Dynamic binary translation for sgx enclaves,” *ACM Transactions on Privacy and Security*, vol. 25, no. 4, pp. 1–40, 2022.
- [21] S. Shinde, D. Le Tien, S. Tople, and P. Saxena, “Panoply: Low-tcb linux applications with sgx enclaves,” in *NDSS*, 2017.
- [22] MICROSOFT, “Open enclave sdk,” <https://openenclave.io/sdk/>.
- [23] ASYLO, “Asylo: An open and flexible framework for enclave applications,” <https://asylo.dev/>.
- [24] T. Cloosters, J. Willbold, T. Holz, and L. Davi, “SGXFuzz: Efficiently synthesizing nested structures for sgx enclave fuzzing,” in *USENIX Security*, 2022, pp. 3147–3164.
- [25] A. Khan, M. Zou, K. Kim, D. Xu, A. Bianchi, and D. J. Tian, “Fuzzing sgx enclaves via host program mutations,” in *IEEE EuroS&P*, 2023, pp. 472–488.
- [26] D. Yu, J. Wang, H. Fang, Y. Fang, and Y. Zhang, “SEnFuzzer: Detecting sgx memory corruption via information feedback and tailored interface analysis,” in *RAID*, 2023, pp. 485–498.
- [27] R. Cui, L. Zhao, and D. Lie, “Emilia: Catching iago in legacy code,” in *NDSS*, 2021.
- [28] T. Cloosters, M. Rodler, and L. Davi, “TeeRex: Discovery and exploitation of memory corruption vulnerabilities in sgx enclaves,” in *USENIX Security*, 2020, pp. 841–858.
- [29] P. Antonino, W. A. Woloszyn, and A. Roscoe, “Guardian: Symbolic validation of orderliness in sgx enclaves,” in *CCSW*, 2021, pp. 111–123.
- [30] Y. Wang, Z. Zhang, N. He, Z. Zhong, S. Guo, Q. Bao, D. Li, Y. Guo, and X. Chen, “Symgx: Detecting cross-boundary pointer vulnerabilities of sgx applications via static symbolic execution,” in *ACM SIGSAC CCS*, 2023, pp. 2710–2724.
- [31] F. Alder, L.-A. Daniel, D. Oswald, F. Piessens, and J. Van Bulck, “Pandora: Principled symbolic validation of intel sgx enclave runtimes,” in *IEEE (S&P)*, 2024.
- [32] F. Toffalini, M. Payer, J. Zhou, and L. Cavallaro, “Designing a provenance analysis for sgx enclaves,” in *ACSAC*, 2022, pp. 102–116.
- [33] S. Chen, Z. Lin, and Y. Zhang, “Controlled data races in enclaves: Attacks and detection,” in *USENIX Security*, 2023, pp. 4069–4086.
- [34] S. Checkoway and H. Shacham, “Iago attacks: Why the system call api is a bad untrusted rpc interface,” *ACM SIGARCH Computer Architecture News*, vol. 41, no. 1, pp. 253–264, 2013.
- [35] P. Guide, “Intel® 64 and ia-32 architectures software developer’s manual,” *Volume 3B: System programming Guide, Part*, vol. 2, no. 11, pp. 0–40, 2011.
- [36] “LibFuzzer,” <https://lvm.org/docs/LibFuzzer.html>.
- [37] R. Swiecki, “Honggfuzz,” <http://code.google.com/p/honggfuzz>.
- [38] Intel, “How to run intel® software guard extensions’ simulation mode,” <https://www.intel.com/content/www/us/en/developer/articles/training/usage-of-simulation-mode-in-sgx-enhanced-application.html>.
- [39] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, “Efficient software-based fault isolation,” in *SOSP*, 1993, pp. 203–216.
- [40] rscosta, “Sgxcryptofile - app for encrypting and decrypting files using intel sgx,” <https://github.com/rscosta/SGXCryptoFile>.
- [41] Nazmus-Sadat, “Trusted hardware assisted genome-wide association studies,” <https://github.com/Nazmus-Sadat/sgx-gwas>.
- [42] sparkly9399, “Dfc string pattern matching library for sgx application,” <https://github.com/sparkly9399/SGX-DFC>.
- [43] Dsc-sgx, “sgx-kmeans,” <https://github.com/dsc-sgx/sgx-kmeans>.
- [44] Asonnino, “Simple password-wallet application based on intel sgx for linux,” <https://github.com/asonnino/sgx-wallet>.
- [45] jefflora, “guess-the-number,” <https://github.com/jefflora/sgx-demo-app/tree/master/guess-the-number>.
- [46] Maxul, “Aes-ni crypto for intel sgx,” <https://github.com/Maxul/SGX-AES-256>.
- [47] utds3lab, “The nbench benchmark ported to sgx,” <https://github.com/utds3lab/sgx-nbench>.
- [48] LeoneChen, “SwitchlessSGXsan,” <https://github.com/LeoneChen/SwitchlessSGXSan>.
- [49] occlum, “Sampleenclave,” <https://github.com/occlum/linux-sgx/tree/cd4e96fd2854536b8ad36d58b4a5554203cebf33/SampleCode/SampleEnclave>.
- [50] lemonviv, “Sgx-darknet: Sgx compatible ml library,” <https://github.com/lemonviv/sgx-dnet>.
- [51] SabaEskandarian, “Port of webassembly interpreter to run inside an sgx enclave,” <https://github.com/SabaEskandarian/webassemblyEnclave>.
- [52] tyurek, “sgxvpn,” <https://github.com/tyurek/sgxvpn>.
- [53] AmbitionXiang, “Sample for allocating and managing untrusted memory inside an enclave,” <https://github.com/AmbitionXiang/ucmalloc-sgx>.
- [54] donayam1, “Mex sgx,” <https://github.com/donayam1/mex-sgx>.
- [55] R-Niagra, “Bank app built using linux sgx,” <https://github.com/R-Niagra/sgxBankApp>.
- [56] benschreiber, “A lisp interpreter in sgx for use as a secure autograder,” <https://github.com/benschreiber/cave>.
- [57] S. Sen, “Intel-sgx-programs,” <https://github.com/sensatyaki/Intel-SGX-Programs>.
- [58] intel, “RemoteAttestation,” <https://github.com/intel/linux-sgx/tree/main/SampleCode/RemoteAttestation>.
- [59] “The heartb donayam1leed bug,” <https://heartbleed.com/>.
- [60] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, “Address-sanitizer: A fast address sanity checker,” in *USENIX ATC*, 2012, pp. 309–318.
- [61] W. Han, B. Joe, B. Lee, C. Song, and I. Shin, “Enhancing memory error detection for large-scale applications and fuzz testing,” in *NDSS Symposium*, 2018.